

Graphs

CIS 5450, Fall 2026

Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

- **Course logistics**
- Graphs as mathematical objects
- Graphs as relations
- Graph analysis with NetworkX

Upcoming Deadlines

- HW2 due Sept 30th at 10pm (7 days)
- Midterm is Oct 7th, in person (14 days)
 - Practice midterms posted tonight
- OE2 is ongoing – make sure to keep your appointment!
- OE3 signups will start next week

Midterm exam

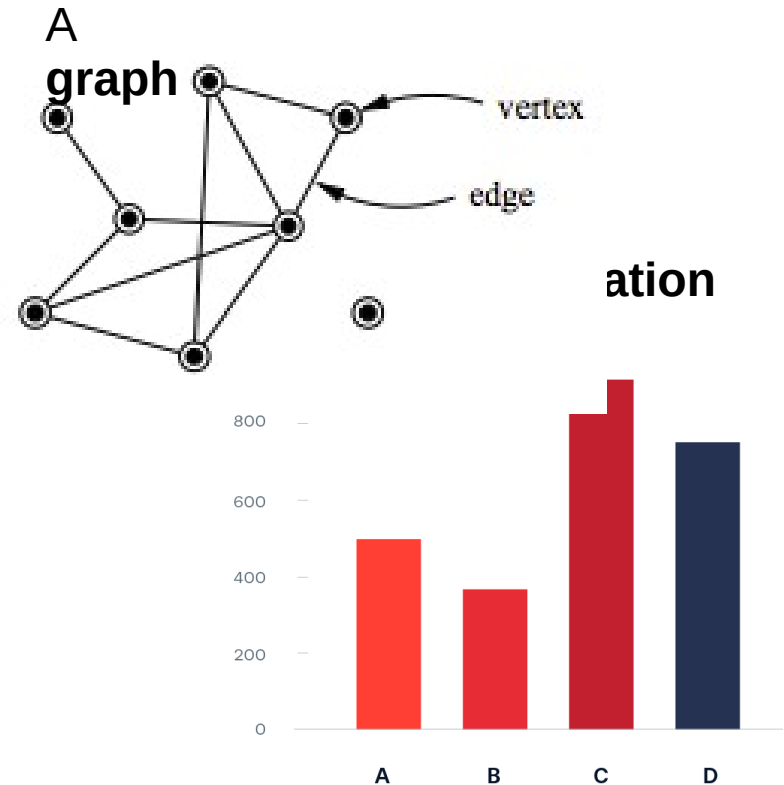
- Class on Oct 5th will be cancelled (study day)
- We will host a few midterm review sessions
 - Details will be posted on Ed
- Topics include everything covered up until the midterm.
- Practice exams posted tonight at 10pm.
 - Same length and format as your exam; different questions.

Outline

- Course logistics
- **Graphs as mathematical objects**
- Graphs as relations
- Graph analysis with NetworkX

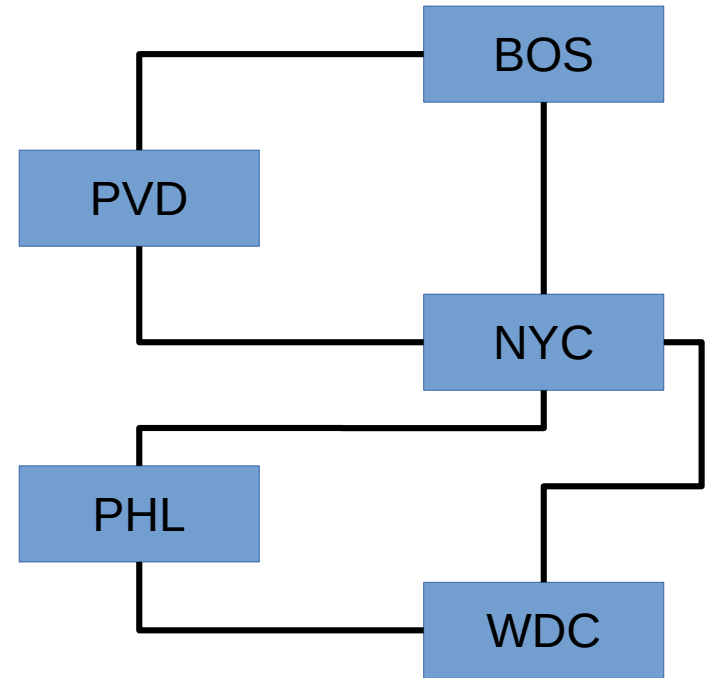
Graphs as math objects

- Overloaded term, can mean
- A visualization or plot
- A mathematical object consisting of vertices and edges



Graphs as math objects

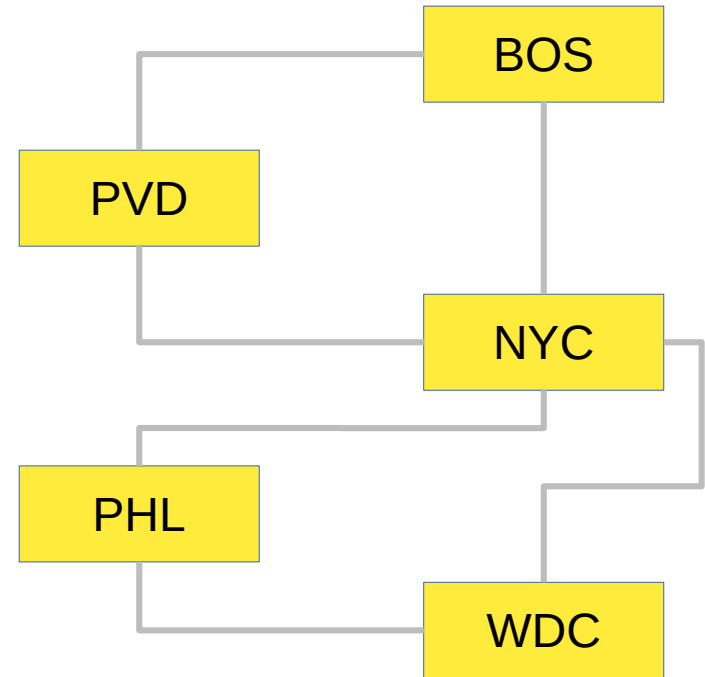
- We define an undirected graph $G = \{V, E\}$
- Where V is a set of **vertices** (singular: vertex) or **nodes**
- Where E is a set of edges ($v_1 \in V, v_2 \in V$)



Graphs as math objects

Vertices / Nodes

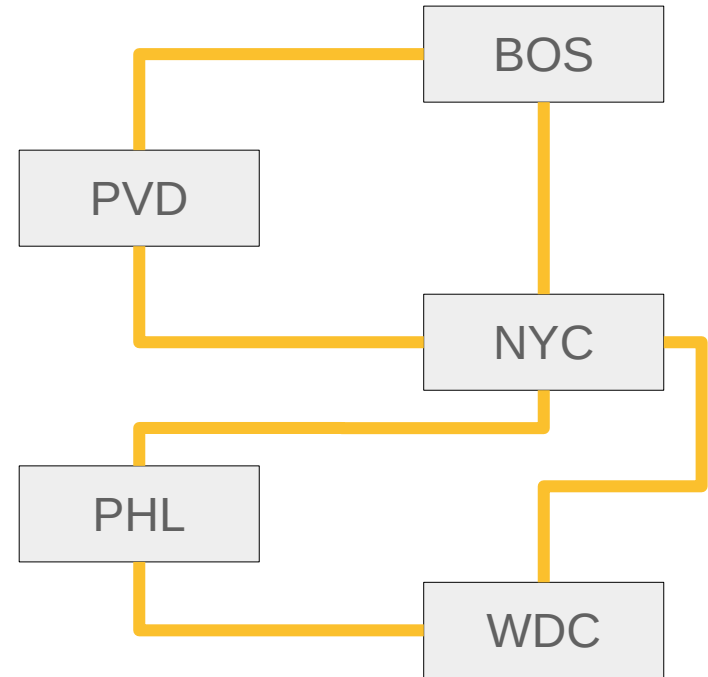
$V = \{\text{"BOS"}, \text{"PVD"},$
 $\text{"NYC"}, \text{"PHL"}, \text{"WDC"}\}$



Graphs as math objects

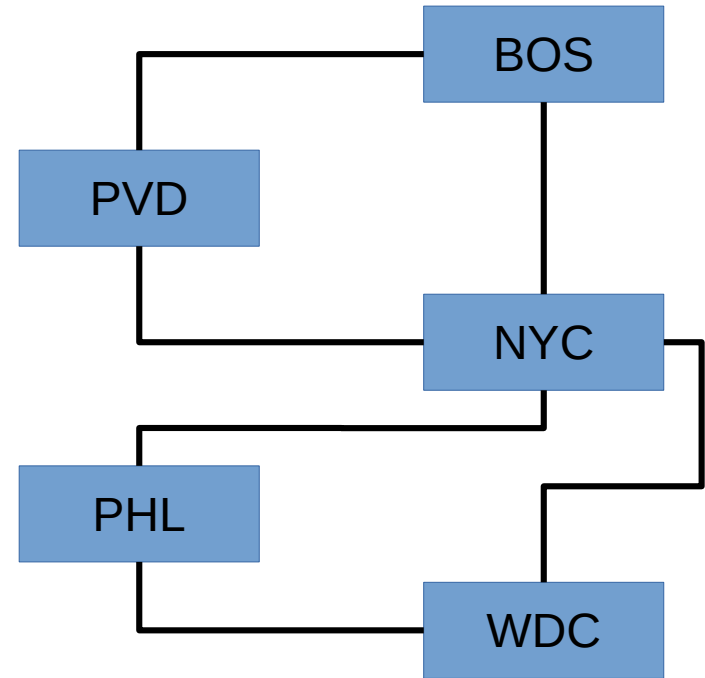
Edges

$E = \{(BOS, PVD), (BOS, NYC), (PVD, NYC), (NYC, PHL), (NYC, WDC), (PHL, WDC)\}$



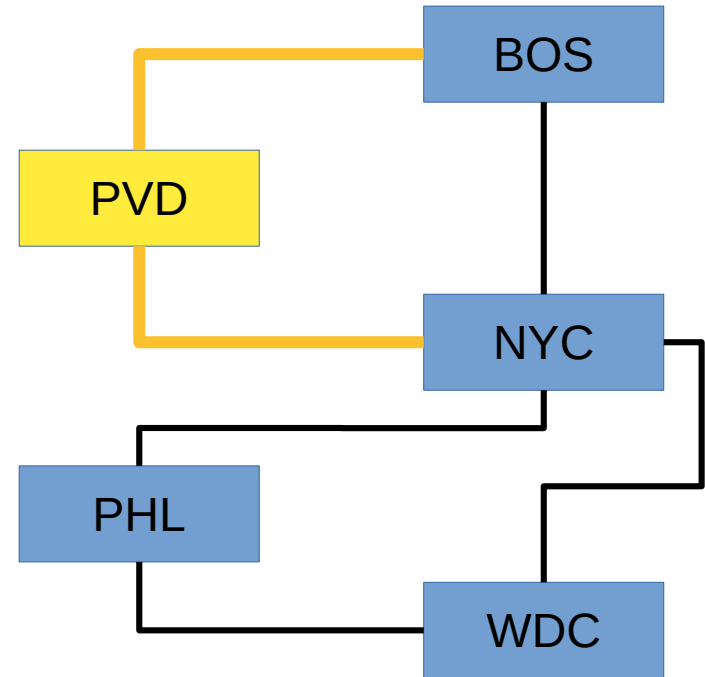
Graphs as math objects

- The **degree** of a node is the number of edges containing that node
- PVD: degree 2
- NYC: degree 4



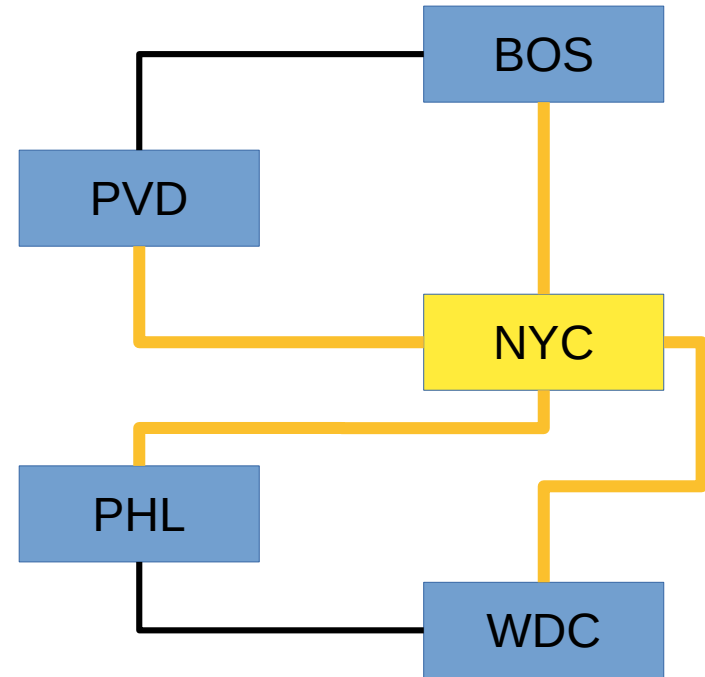
Graphs as math objects

- The **degree** of a node is the number of edges containing that node
- **PVD: degree 2**
- **NYC: degree 4**



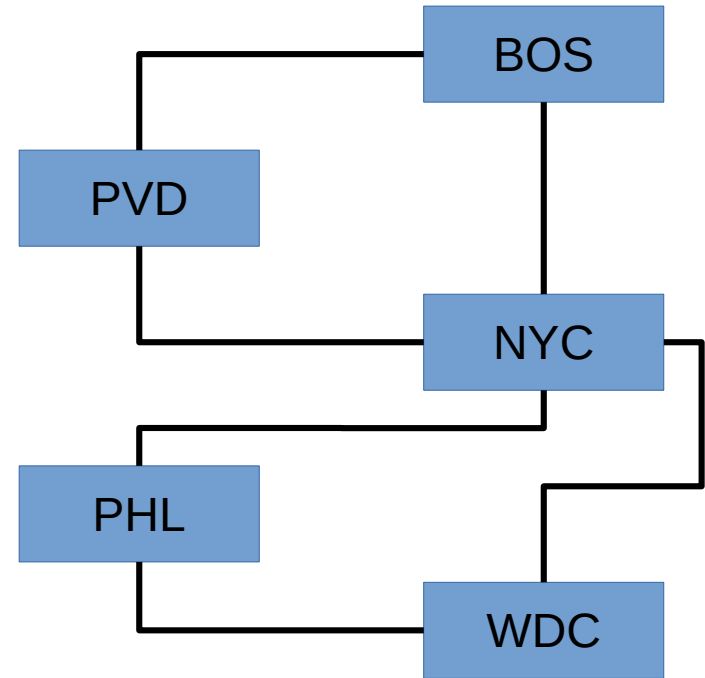
Graphs as math objects

- The **degree** of a node is the number of edges containing that node
- PVD: degree 2
- **NYC: degree 4**



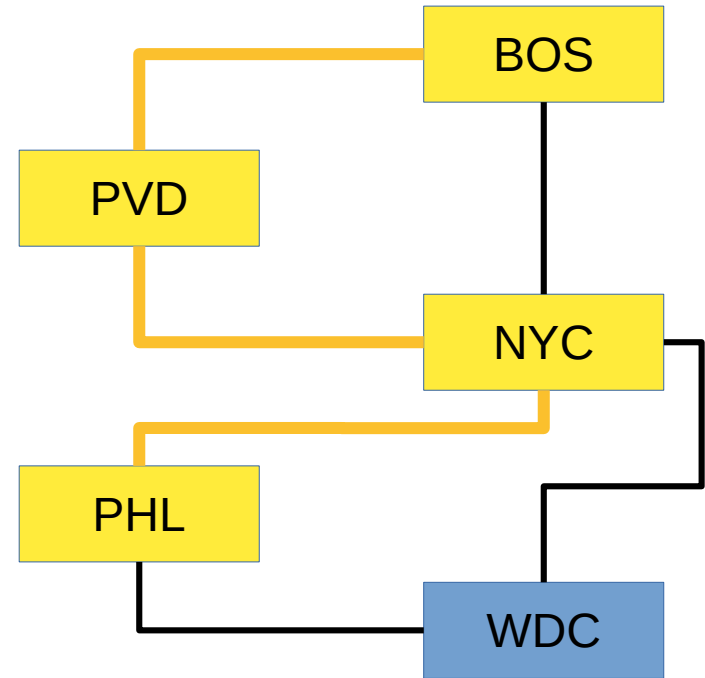
Graphs as math objects

- A **path** is a sequence of directly-connected vertices
- $P_1 = [\text{BOS}, \text{PVD}, \text{NYC}, \text{PHL}]$
- $P_2 = [\text{BOS}, \text{NYC}, \text{PHL}]$



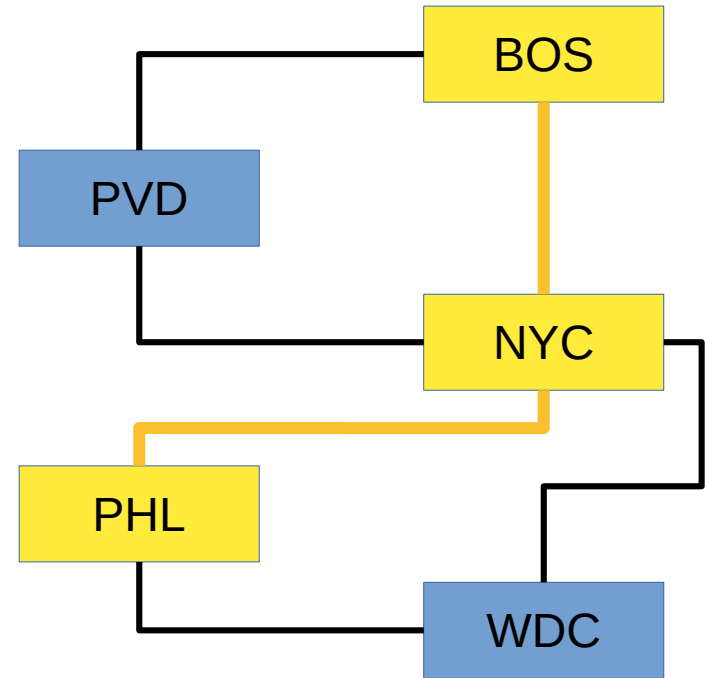
Graphs as math objects

- A **path** is a sequence of directly-connected vertices
- $P_1 = [\text{BOS}, \text{PVD}, \text{NYC}, \text{PHL}]$
- $P_2 = [\text{BOS}, \text{NYC}, \text{PHL}]$



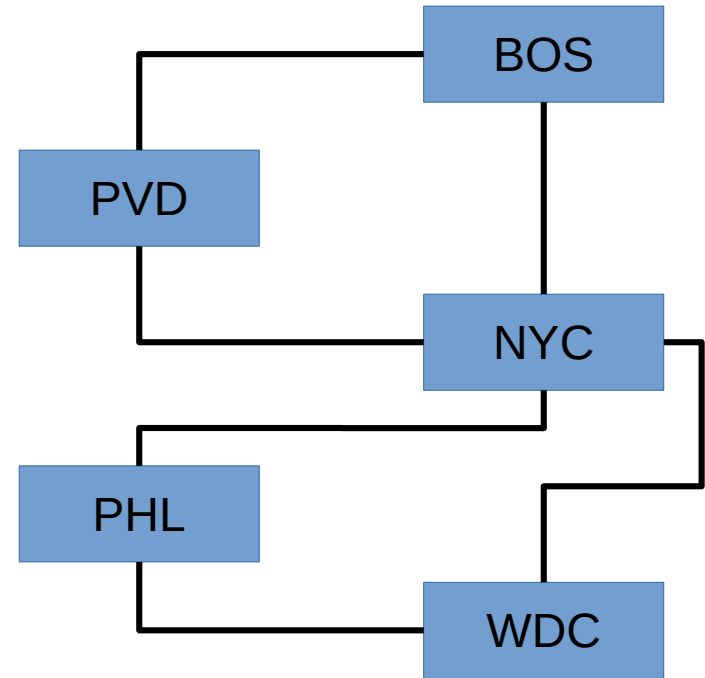
Graphs as math objects

- A **path** is a sequence of directly-connected vertices
- $P_1 = [\text{BOS}, \text{PVD}, \text{NYC}, \text{PHL}]$
- $P_2 = [\text{BOS}, \text{NYC}, \text{PHL}]$



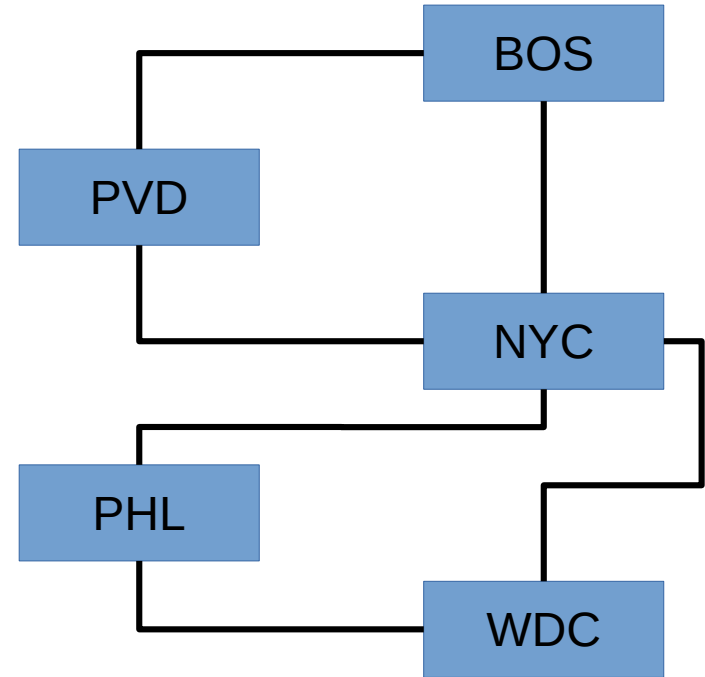
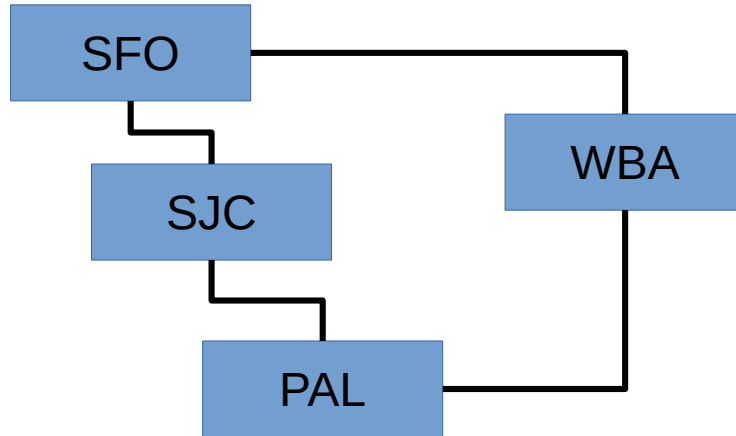
Graphs as math objects

- A graph is **connected** if there a valid path between every node



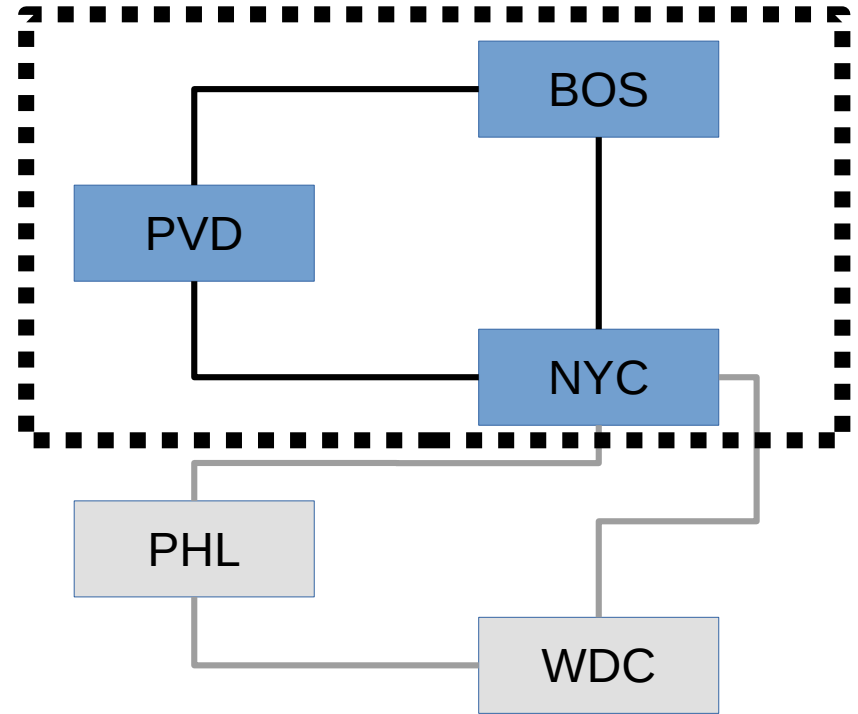
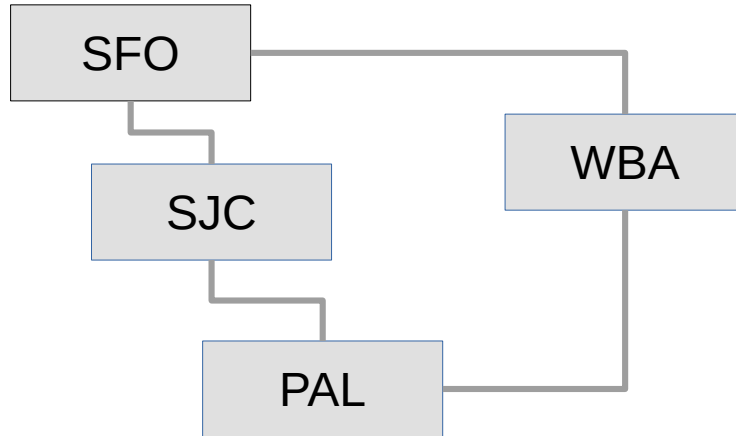
Graphs as math objects

- A graph is **disconnected** if there a pair of vertices with no path between them



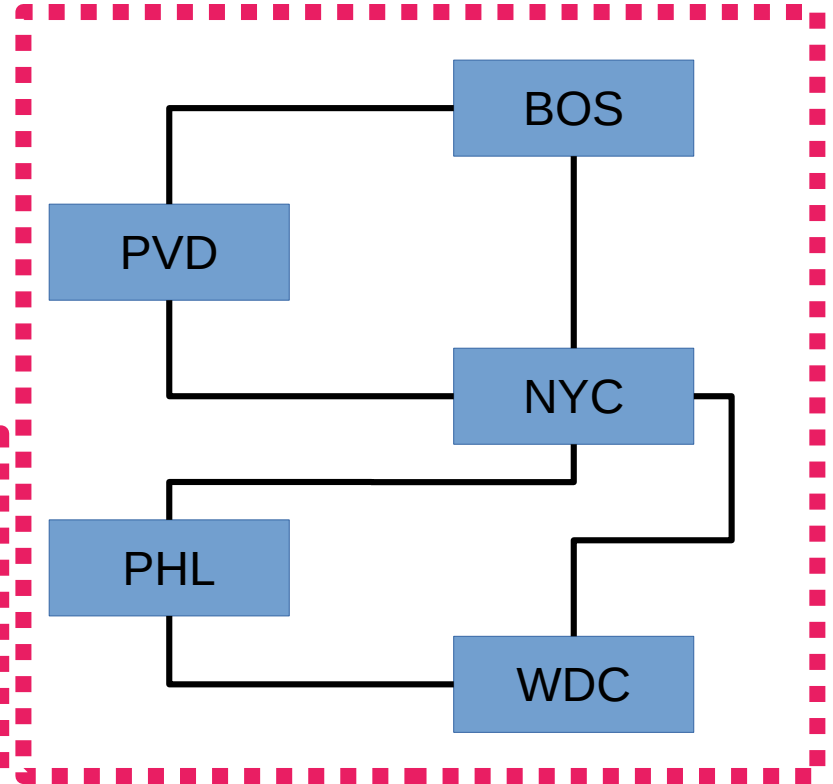
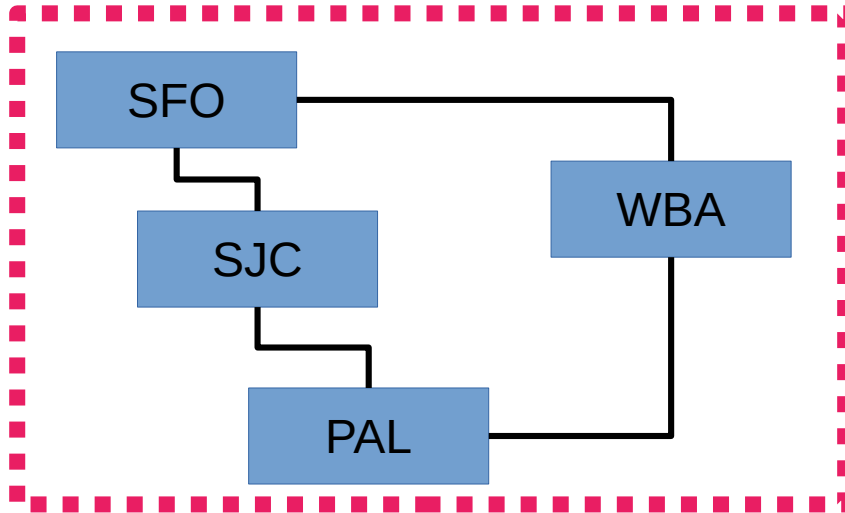
Graphs as math objects

- A **subgraph** A of G is a graph containing a subset of the nodes and edges of G



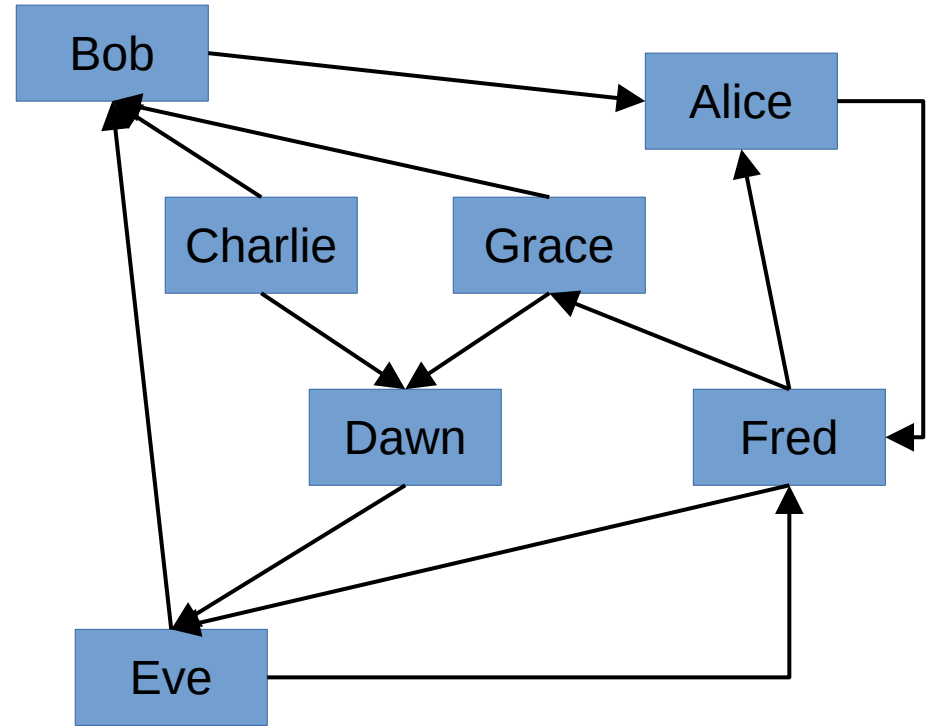
Graphs as math objects

- A **connected component** of G is a maximal connected subgraph of G



Graphs as math objects

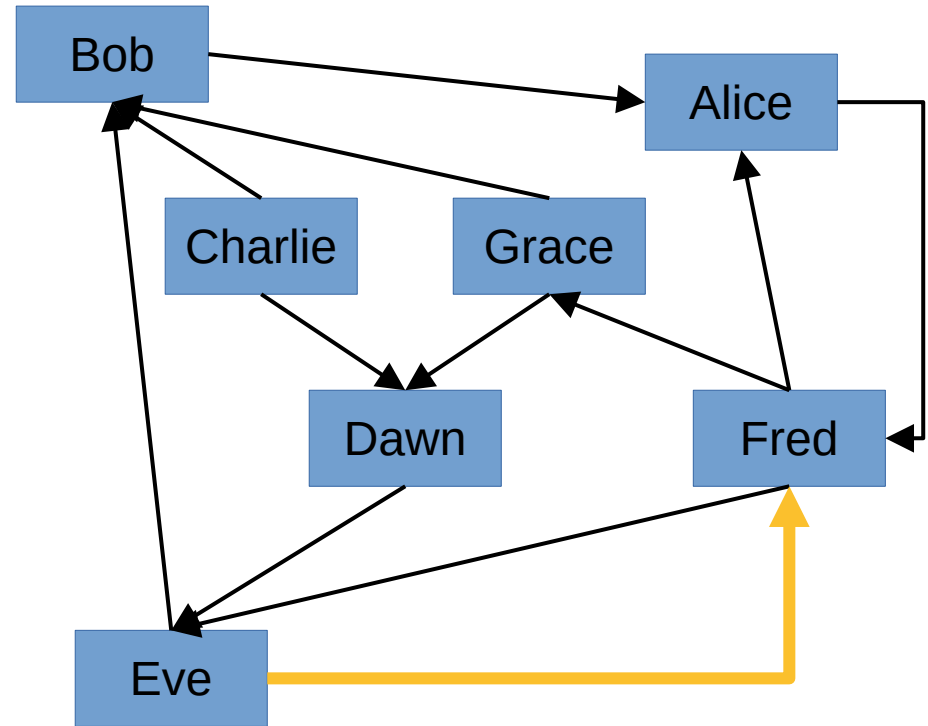
- We define a **directed graph** $D = \{V, E\}$
- Where V is a set of **vertices** (singular: vertex) or **nodes**
- Where E is a set of **ordered edges** $[v_1 \in V, v_2 \in V]$



Graphs as math objects

Directed Edges

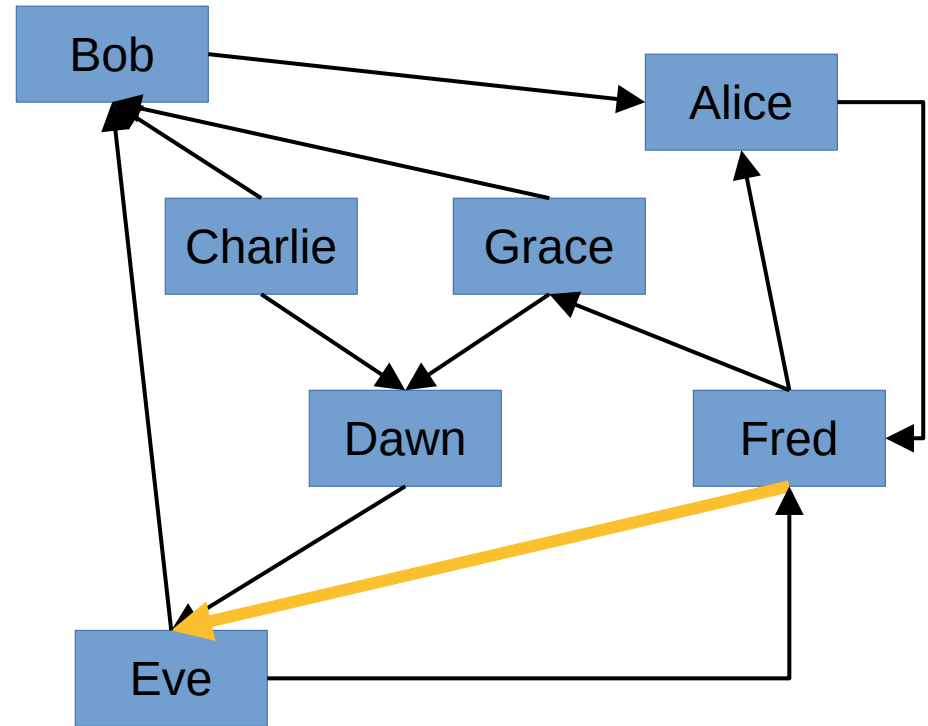
Eve follows Fred



Graphs as math objects

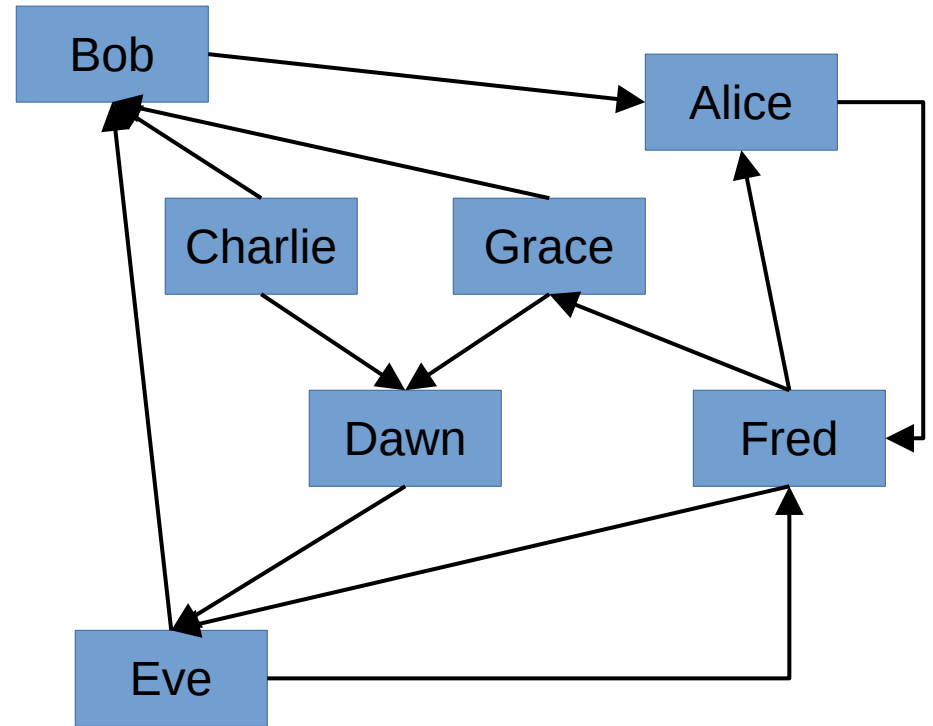
Directed Edges

Fred follows Eve



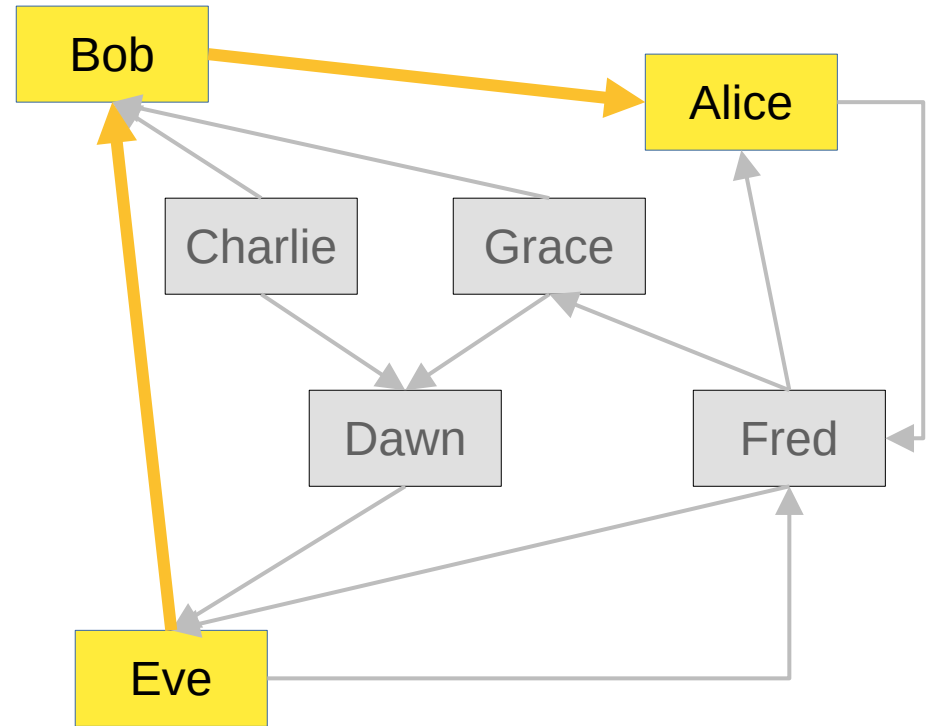
Graphs as math objects

- Paths on directed graphs are similar
- P1: [Eve, Bob, Alice]
- P2: [Grace, Dawn, Eve]



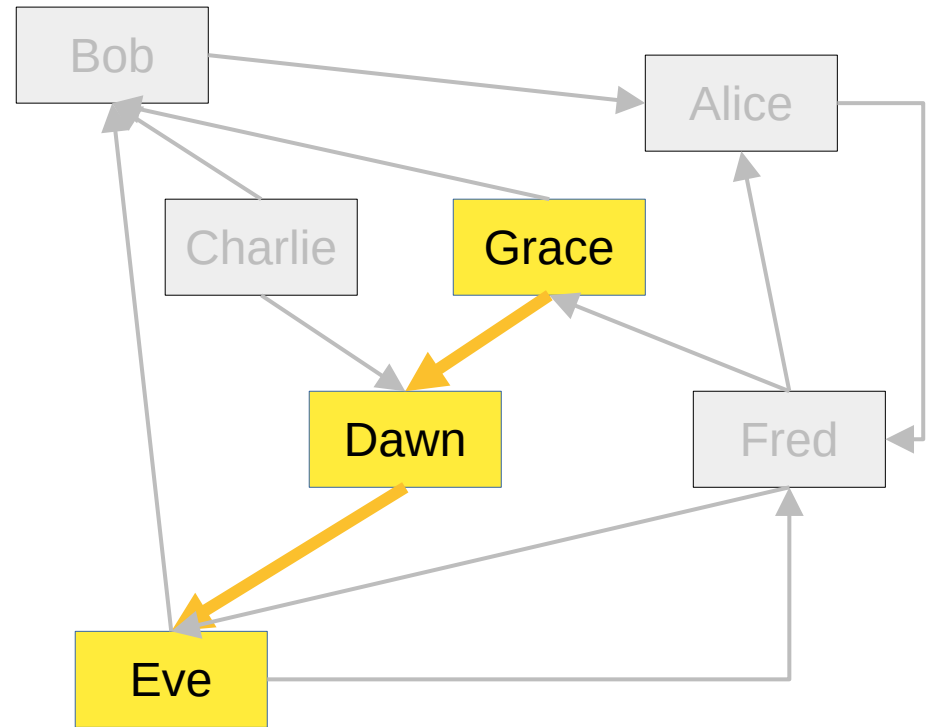
Graphs as math objects

- Paths on directed graphs are similar
- **P1: [Eve, Bob, Alice]**
- P2: [Grace, Dawn, Eve]



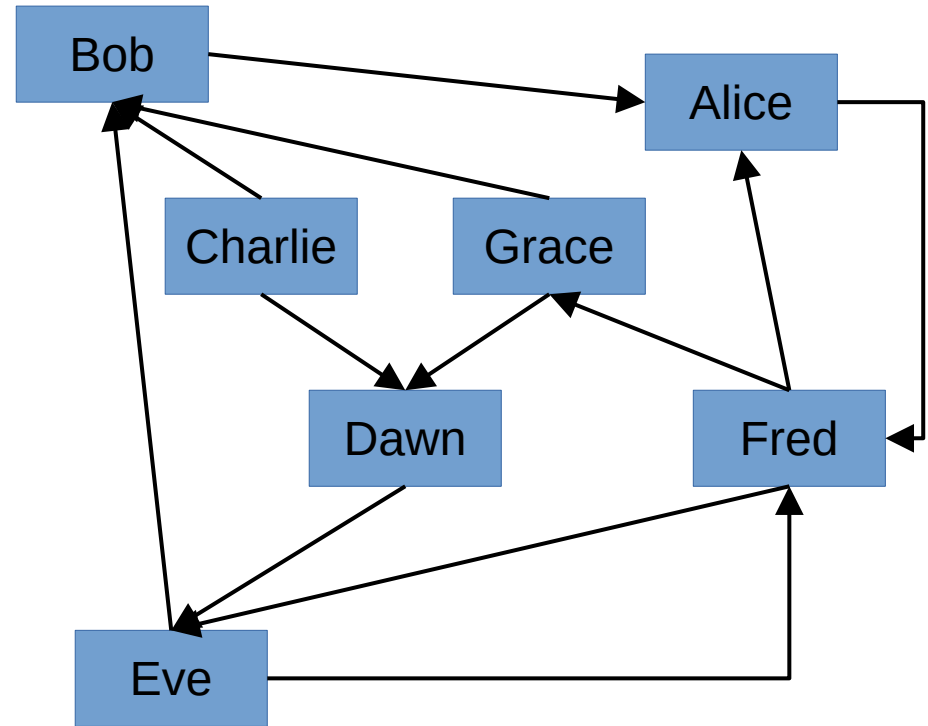
Graphs as math objects

- Paths on directed graphs are similar
- P1: [Eve, Bob, Alice]
- **P2: [Grace, Dawn, Eve]**



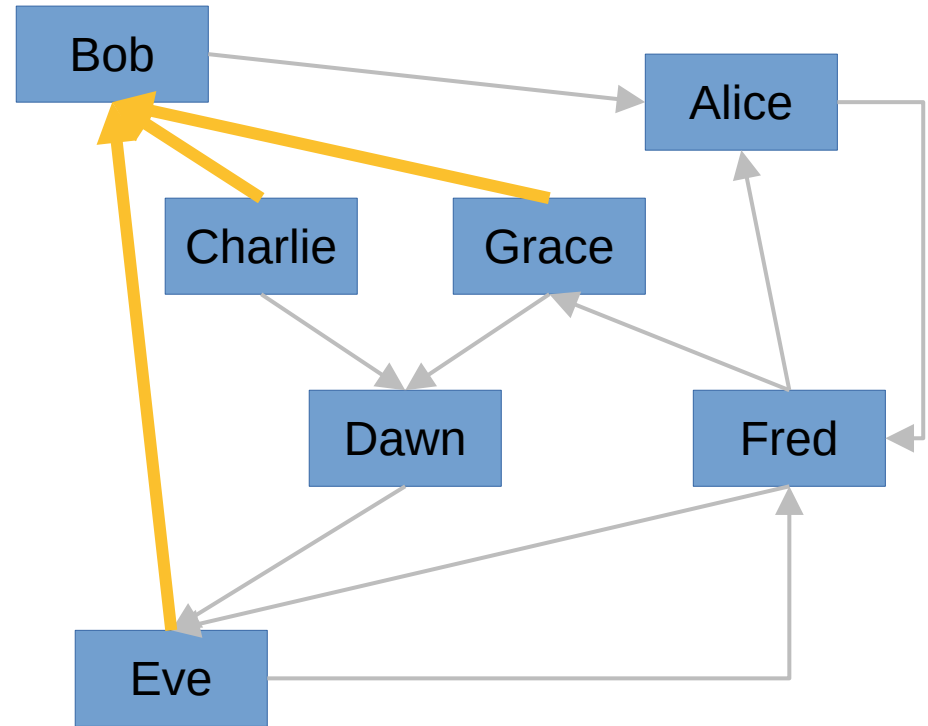
Graphs as math objects

- The **in-degree** of a node is the number of incoming edges
- Bob: in-degree 3
- The **out-degree** of a node is the number of outgoing edges
- Eve: out-degree 2



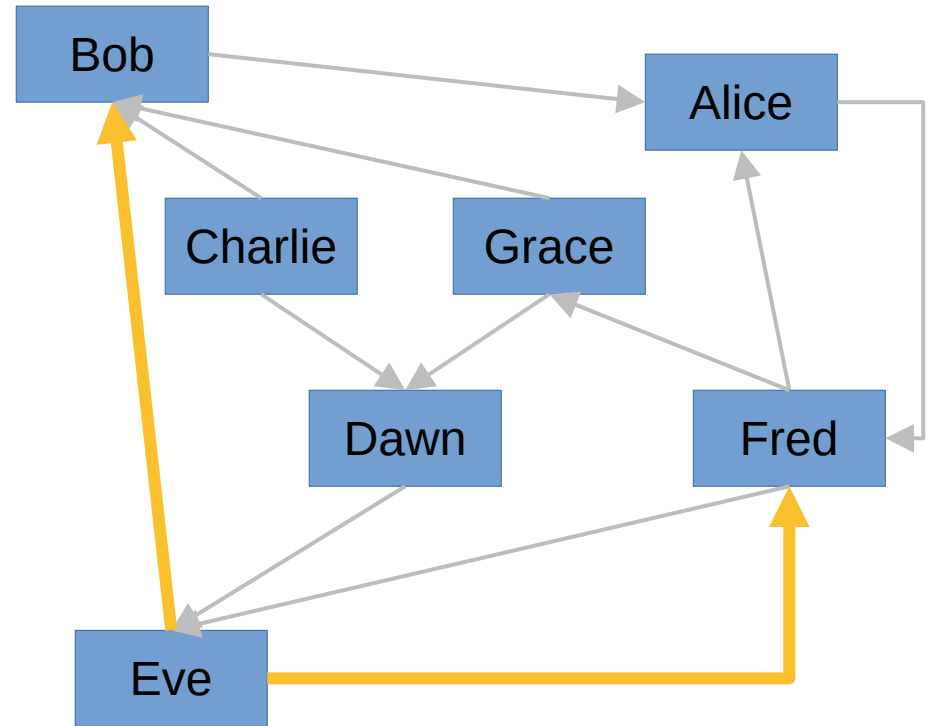
Graphs as math objects

- The **in-degree** of a node is the number of incoming edges
- **Bob: in-degree 3**
- The **out-degree** of a node is the number of outgoing edges
- Eve: out-degree 2



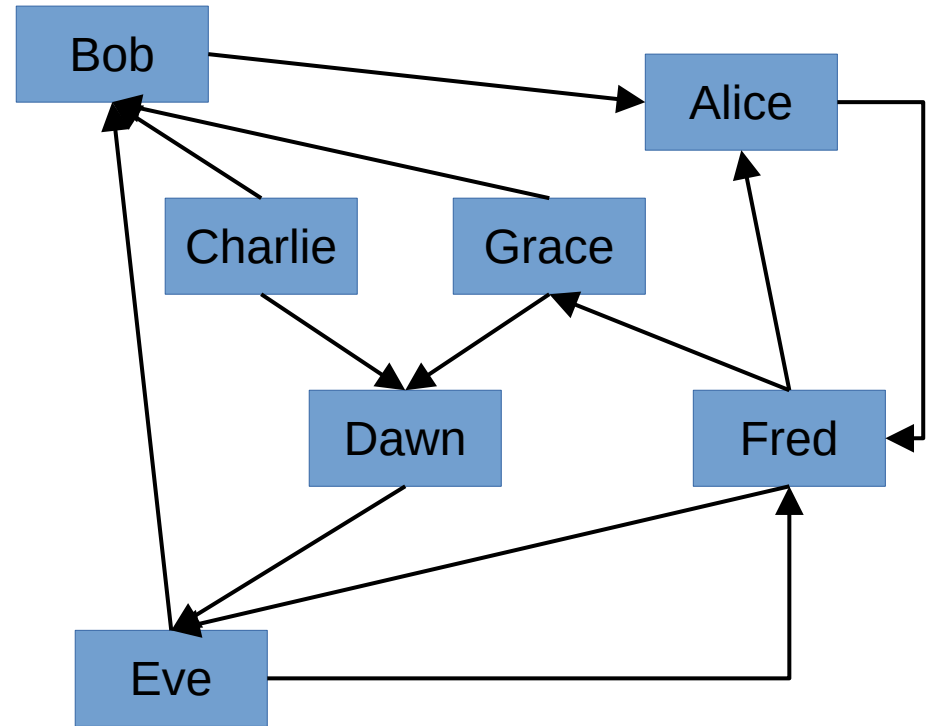
Graphs as math objects

- The **in-degree** of a node is the number of incoming edges
- Bob: in-degree 3
- The **out-degree** of a node is the number of outgoing edges
- Eve: out-degree 2



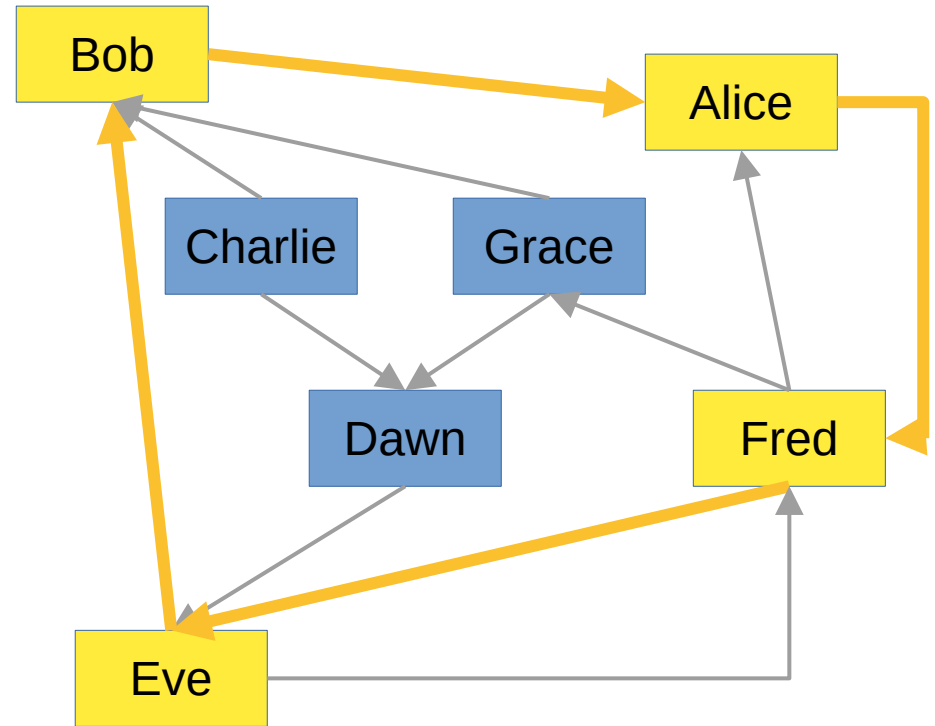
Graphs as math objects

- A directed graph is **cyclic** if we can find a path that starts and stops at the same node



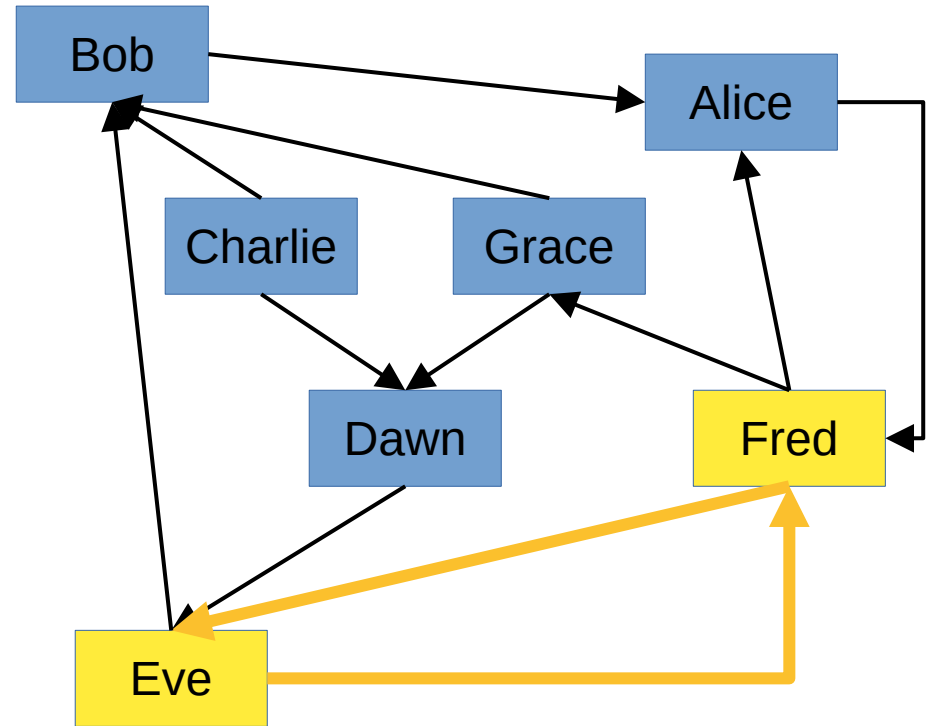
Graphs as math objects

- A directed graph is **cyclic** if we can find a path that starts and stops at the same node



Graphs as math objects

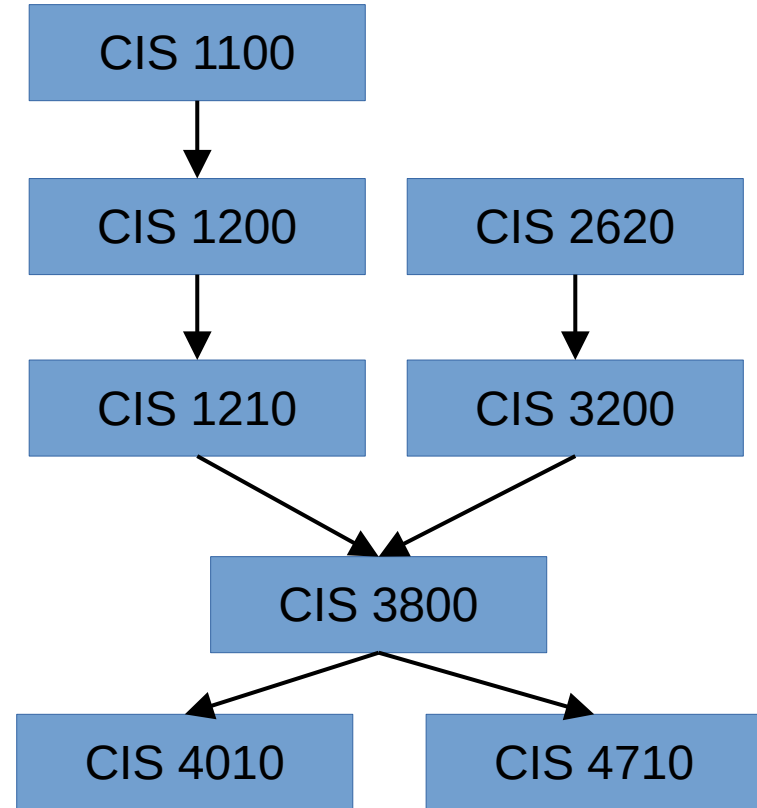
- A directed graph is **cyclic** if we can find a path that starts and stops at the same node



Graphs as math objects

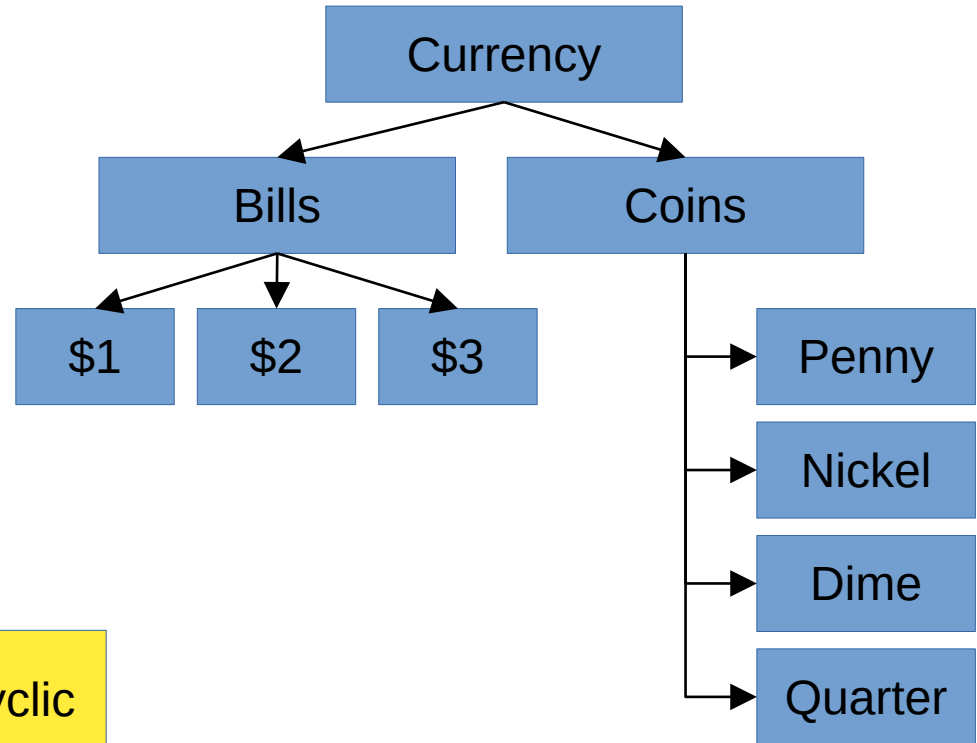
- A directed graph is **acyclic** if it is not cyclic

Ex: dependency graph. What would a cycle mean in a graph like this?



Graphs as math objects

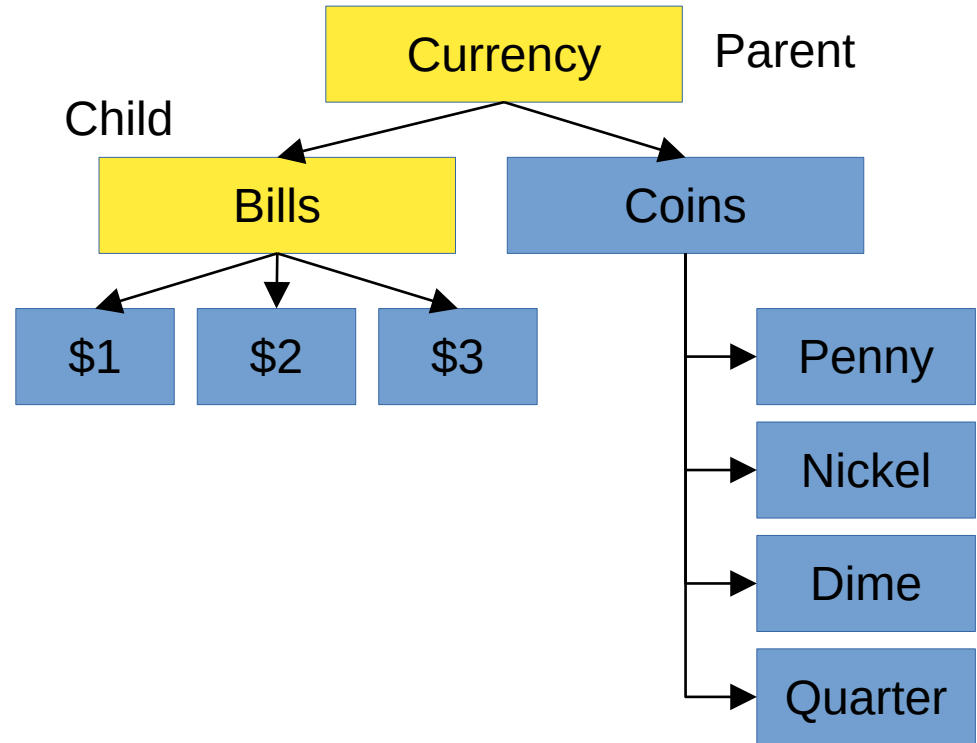
- A directed graph is **tree** if all of the following hold:
- There is one node with in-degree = zero
- Every other node has in-degree = one
- It is acyclic



Why does acyclic matter?

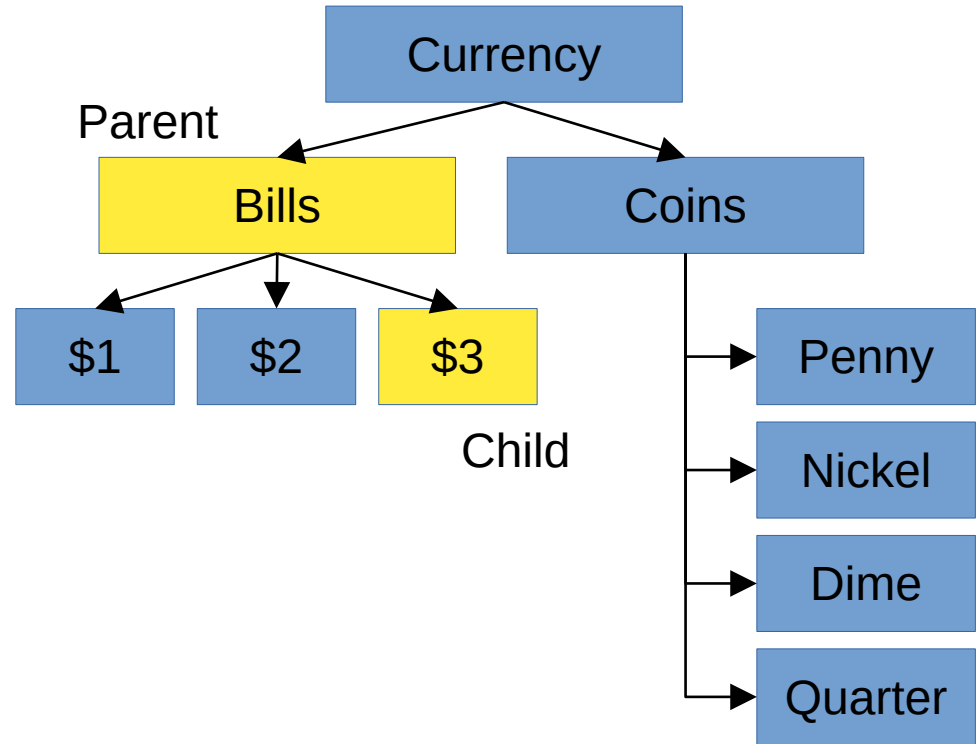
Graphs as math objects

- We sometimes refer to **parent** nodes and **child** nodes



Graphs as math objects

- We sometimes refer to **parent** nodes and **child** nodes

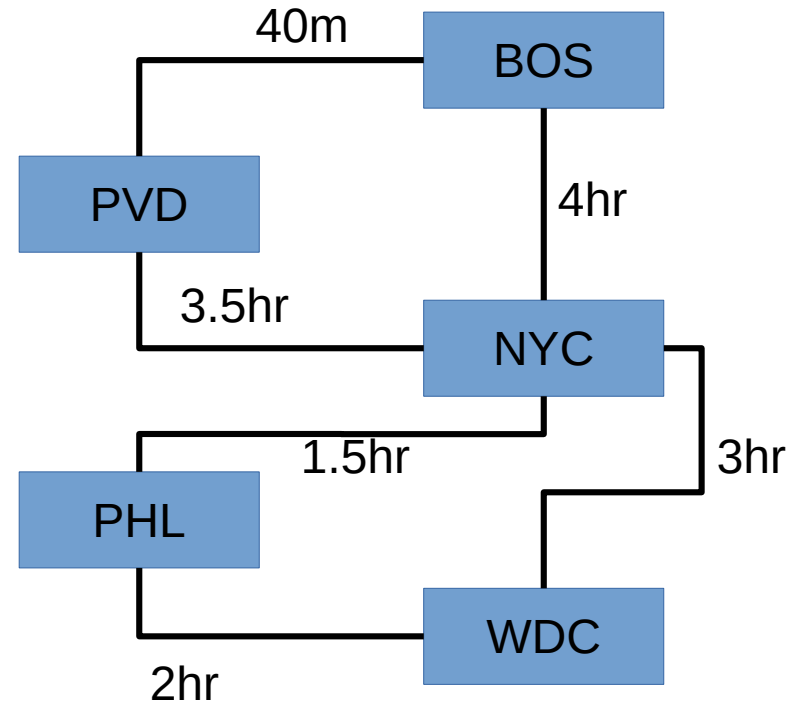


Graphs as math objects

- A directed or undirected graph can potentially be **edge-weighted**

- There is a function

$$W(e \in E) \rightarrow \mathbb{R}$$



Graphs as math objects

- A directed or undirected graph can potentially be **edge-weighted**
- There is a function

$$W(e \in E) \rightarrow \mathbb{R}$$

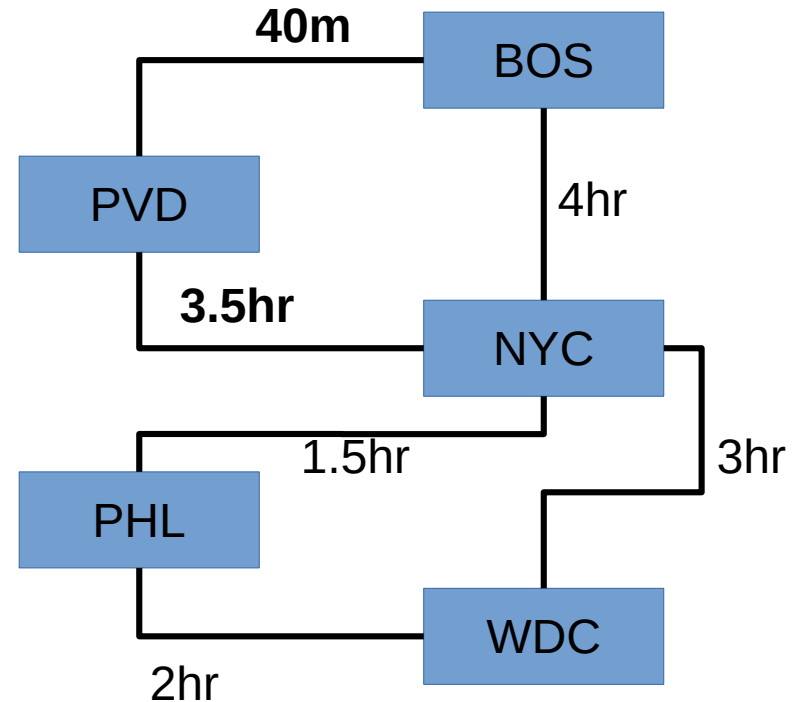
The **cost** or **length** of a path is the sum of the edge weights

P1 = BOS, PVD, NYC

BOS → PVD: 40m

PVD → NYC: 3.5hrs

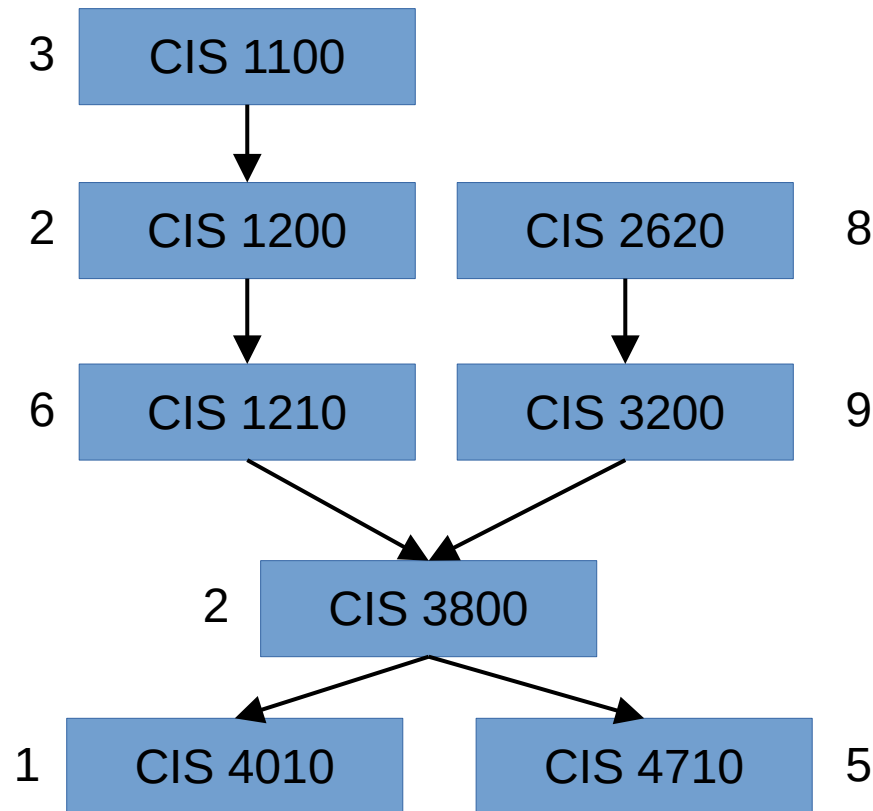
P1 cost: 4 hours 10 minutes



Graphs as math objects

- A directed or undirected graph can potentially be **vertex-weighted**
- There is a function

$$W(v \in V) \rightarrow \mathbb{R}$$

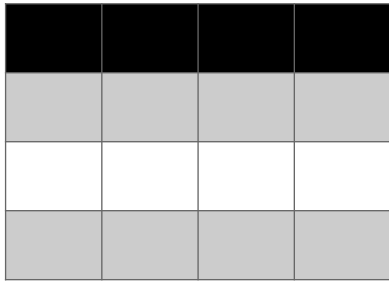


Outline

- Course logistics
- Graphs as mathematical objects
- **Graphs as relations**
- Graph analysis with NetworkX

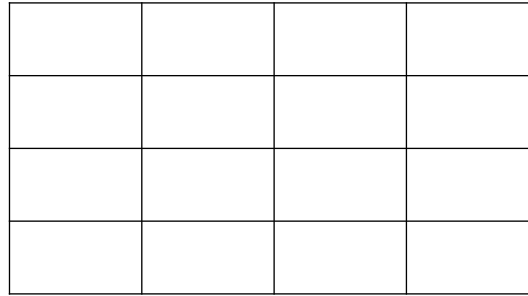
Data Modalities

- “Modality” = form, format, type, etc.



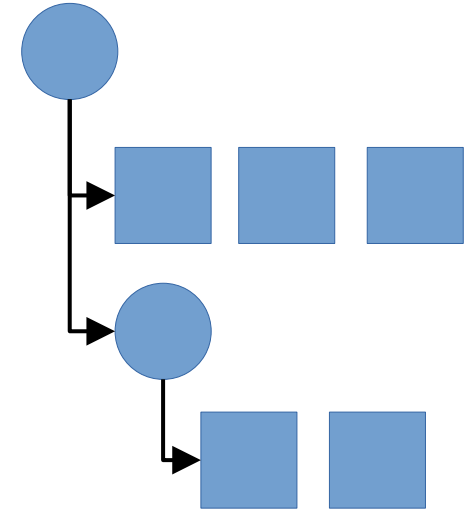
Tables

Relations
Spreadsheets
Dataframes



Arrays

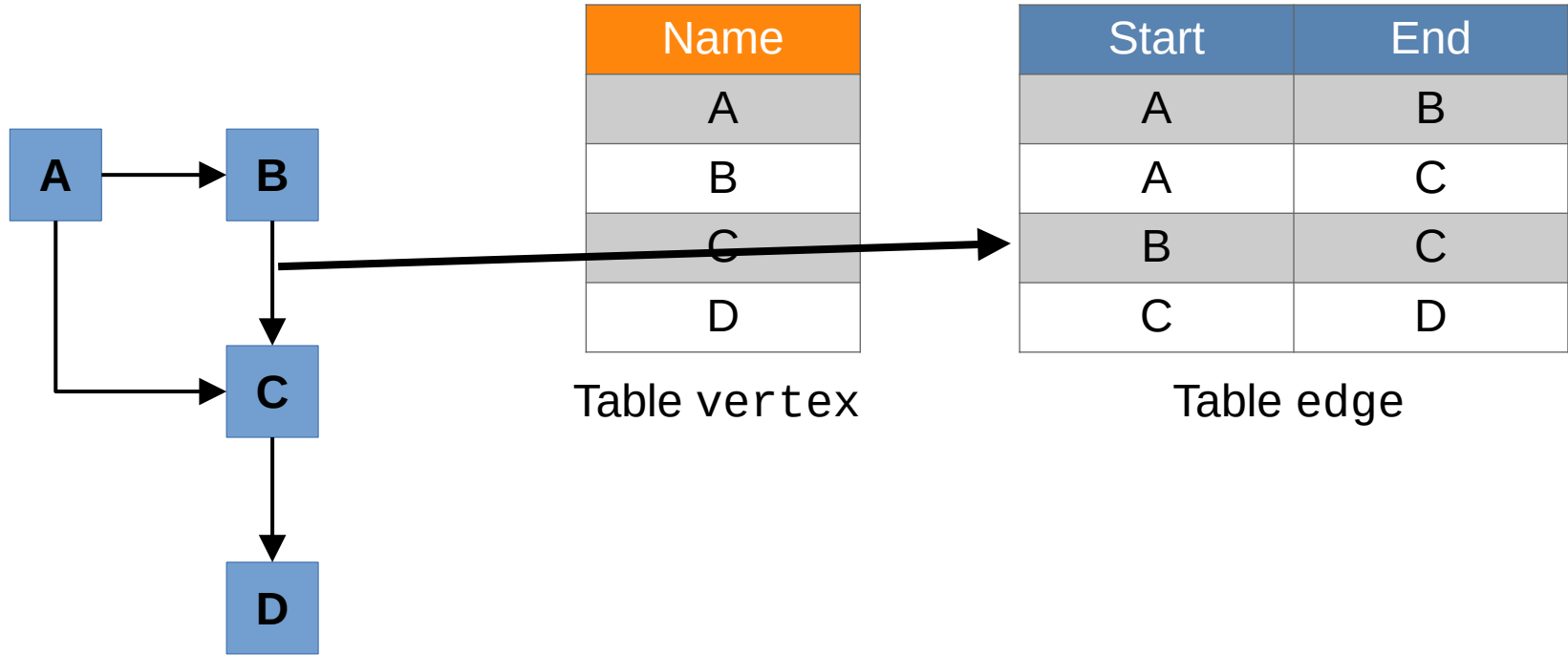
Tensors
Matrix



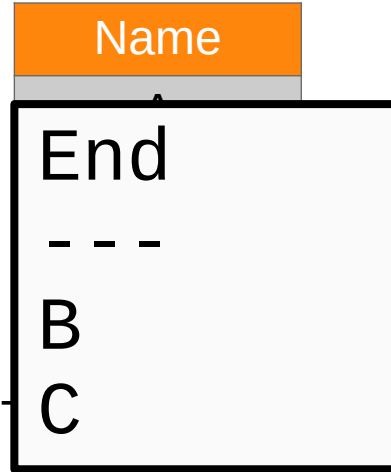
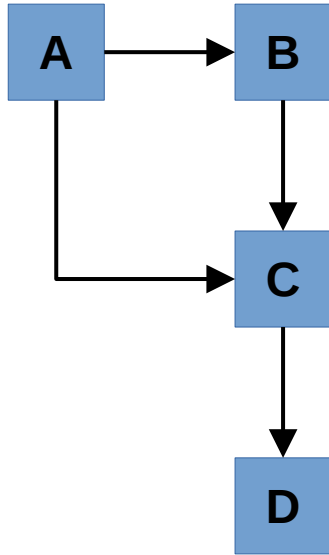
Document

Hierarchy
Tree
HTML / XML

Graphs as Relations



Graphs as Relations



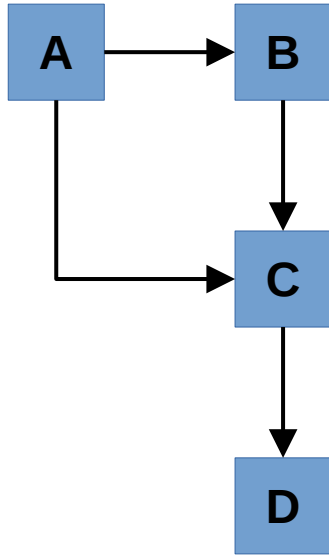
Start	End
A	B
A	C
B	C
C	D

Table edge

Find all nodes with an edge from A

```
SELECT edge.End
FROM   edge
WHERE  edge.Start = 'A';
```

Graphs as Relations



Name	
Start	End
-----	---
A	B
A	C

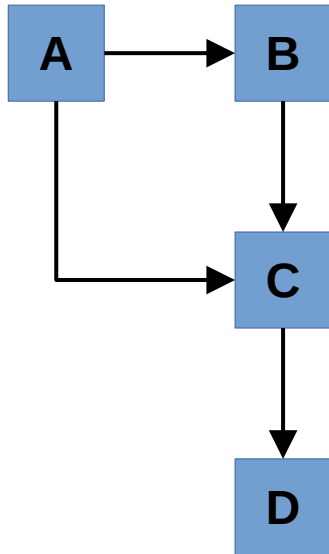
Start	End
A	B
A	C
B	C
C	D

Table edge

Find all paths of length 1 starting from A

```
SELECT edge.Start, edge.End
FROM   edge
WHERE  edge.Start = 'A';
```

Graphs as Relations



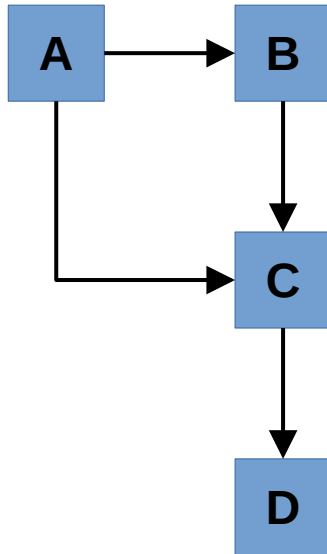
Name			Start	End
Start	End	End		
---	---	---	A	B
A	B	C	A	C
A	C	D	B	C
			C	D

Table edge

Find all paths of length 2 starting from A

```
SELECT e1.Start, e1.End, e2.End
FROM   edge e1, edge e2
WHERE  e1.Start = 'A'
AND    e1.End = e2.Start;
```

Graphs as Relations



Name		Start		End
Start	End	End	End	End
---	---	---	---	
A	B	C	D	B
				C
				C
				D

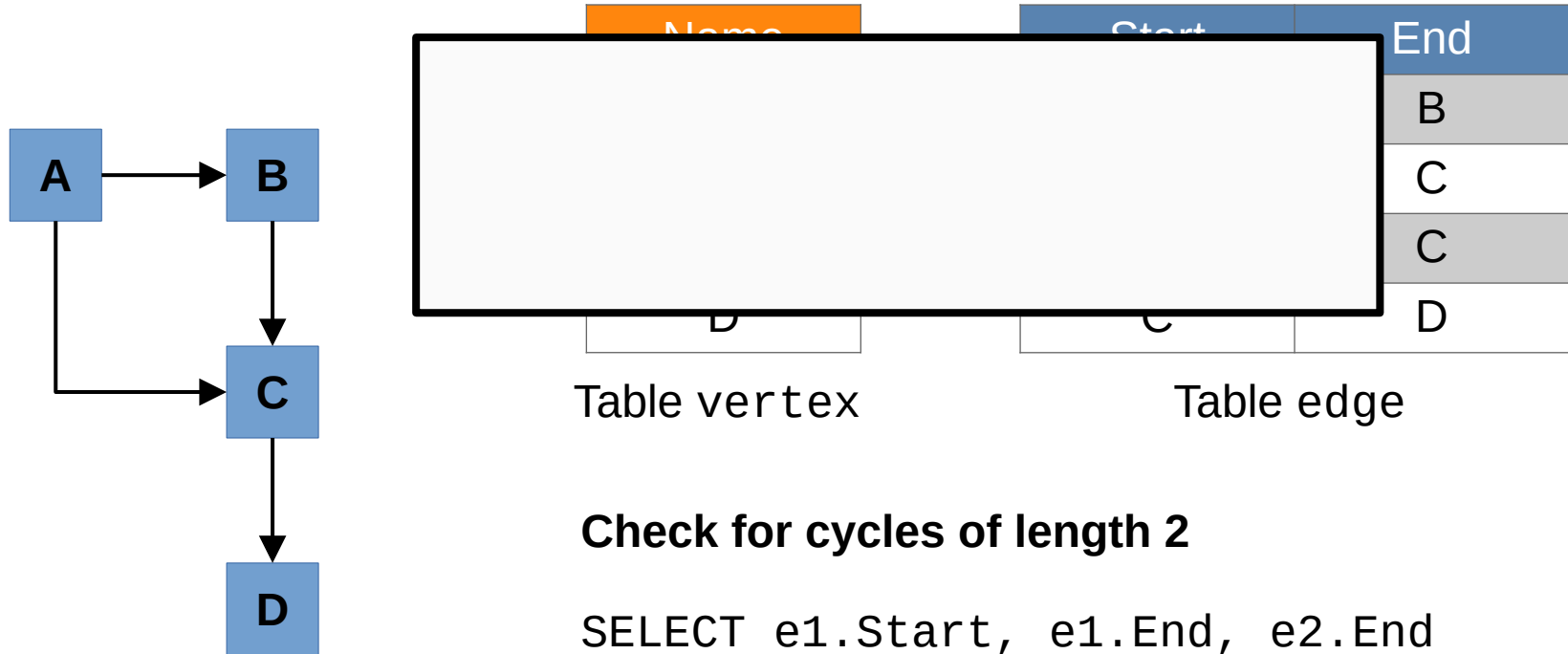
Table vertex

Table edge

Find all paths of length 3 starting from A

```
SELECT e1.Start, e1.End, e2.End, e3.End
FROM   edge e1, edge e2, edge e3
WHERE  e1.Start = 'A'
AND    e1.End = e2.Start
AND    e2.End = e3.Start;
```

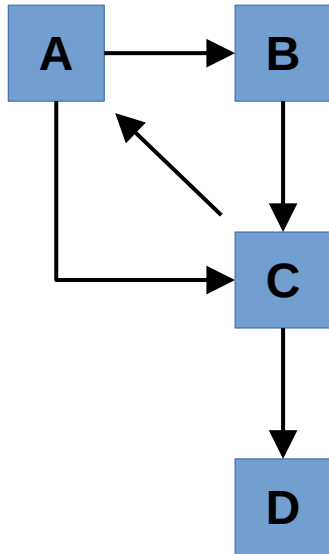
Graphs as Relations



Check for cycles of length 2

```
SELECT e1.Start, e1.End, e2.End
FROM   edge e1, edge e2
WHERE  e1.End = e2.Start
AND    e2.End = e1.Start;
```

Graphs as Relations



Name			Start	End
Start	End	End		
---	---	---		
A	C	A		
C	A	C		

Table vertex

Start	End
A	B
A	C
B	C
C	D

Table edge

Check for cycles of length 2

```

SELECT e1.Start, e1.End, e2.End
FROM   edge e1, edge e2
WHERE  e1.End = e2.Start
AND    e2.End = e1.Start;
  
```

Graphs as Relations

- You can technically do a lot of graph manipulation with SQL
- All connected points (with any edge length)
- Shortest path
- Checks for acyclic / cyclic
- ... but even the SQL superfans will admit it is a little awkward

Outline

- Course logistics
- Graphs as mathematical objects
- Graphs as relations
- **Graph analysis with NetworkX**

NetworkX

- NetworkX is a Python package for graph analysis
- Developed at Los Alamos National lab
- Traditional import: `import networkx as nx`

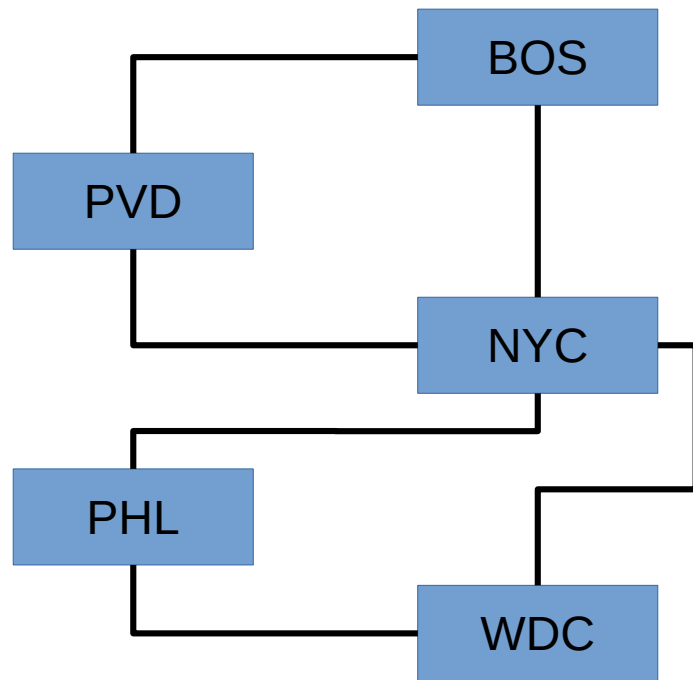
NetworkX

- Let's recreate our train graph

```
[1] import networkx as nx
```



```
G = nx.Graph()
G.add_edge("BOS", "PVD")
G.add_edge("PVD", "NYC")
G.add_edge("BOS", "NYC")
G.add_edge("NYC", "PHL")
G.add_edge("PHL", "WDC")
G.add_edge("NYC", "WDC")
```



NetworkX

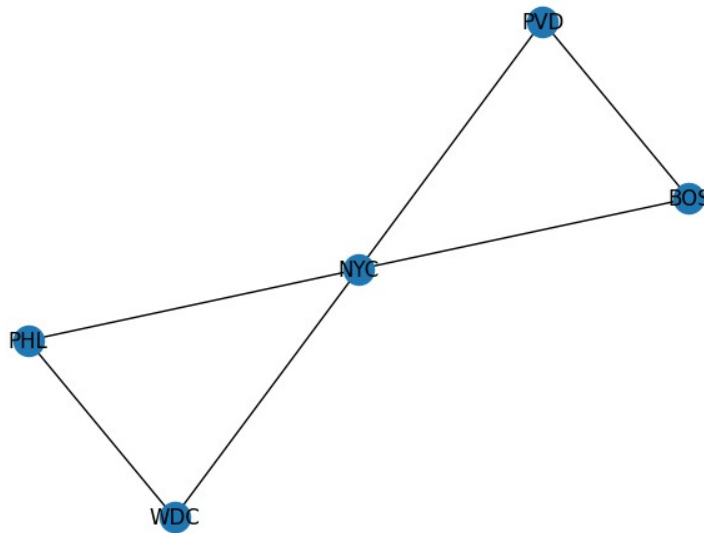
- Let's recreate our train graph

```
[1] import networkx as nx
```



```
G = nx.Graph()
G.add_edge("BOS", "PVD")
G.add_edge("PVD", "NYC")
G.add_edge("BOS", "NYC")
G.add_edge("NYC", "PHL")
G.add_edge("PHL", "WDC")
G.add_edge("NYC", "WDC")

nx.draw(G, with_labels=True)
```



NetworkX

✓
0s

```
[8] nx.is_connected(G)
```

True

✓
0s

```
[13] nx.shortest_path(G, "BOS", "WDC")
```

['BOS', 'NYC', 'WDC']

✓
0s

```
[12] G.degree("-----")
```

2

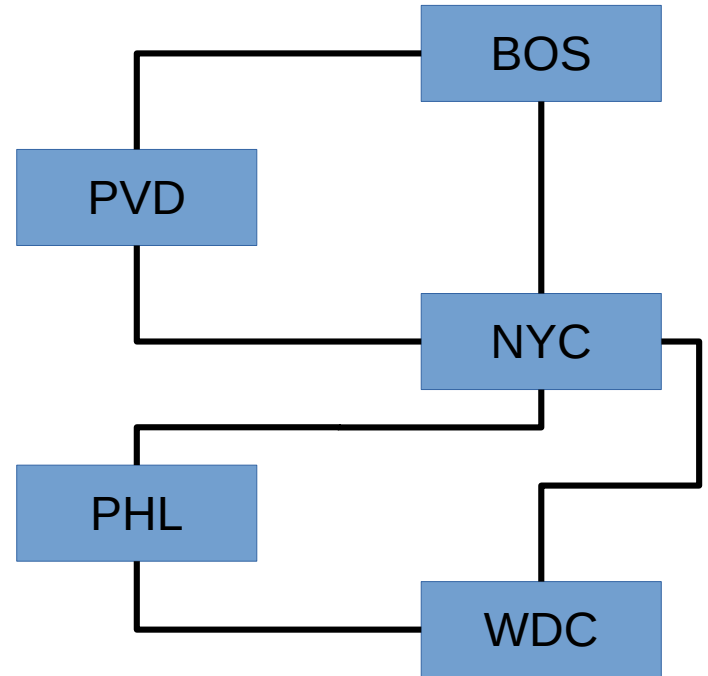
✓
0s

```
[14] nx.diameter(G)
```

2

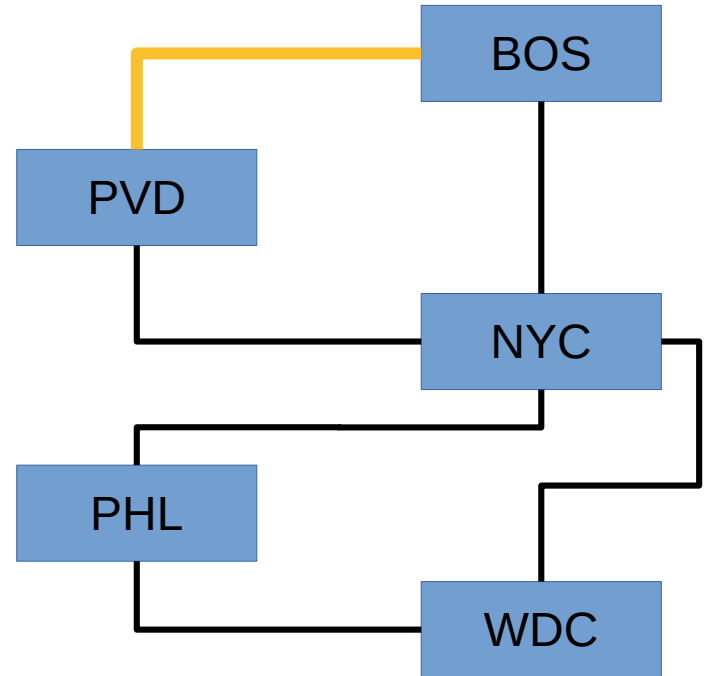
NetworkX

- An Euler circuit is a path through the graph that uses each edge exactly one



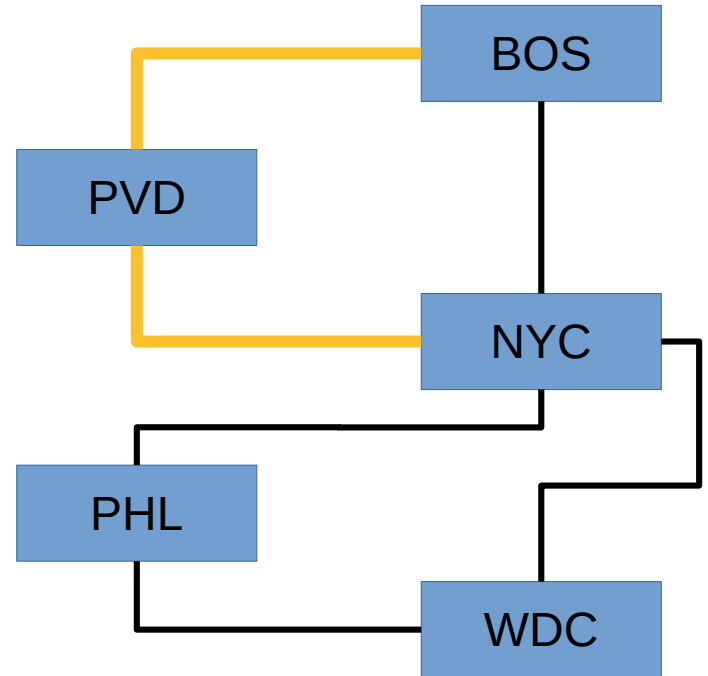
NetworkX

- An Euler circuit is a path through the graph that uses each edge exactly one



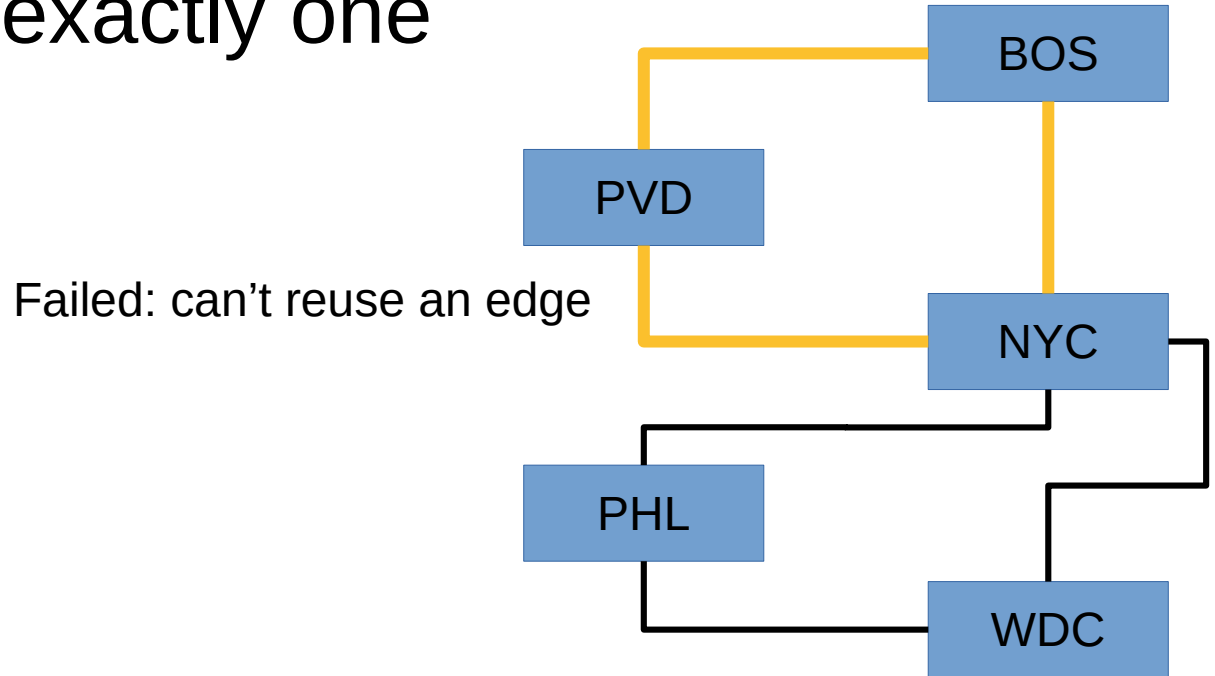
NetworkX

- An Euler circuit is a path through the graph that uses each edge exactly one



NetworkX

- An Euler circuit is a path through the graph that uses each edge exactly one



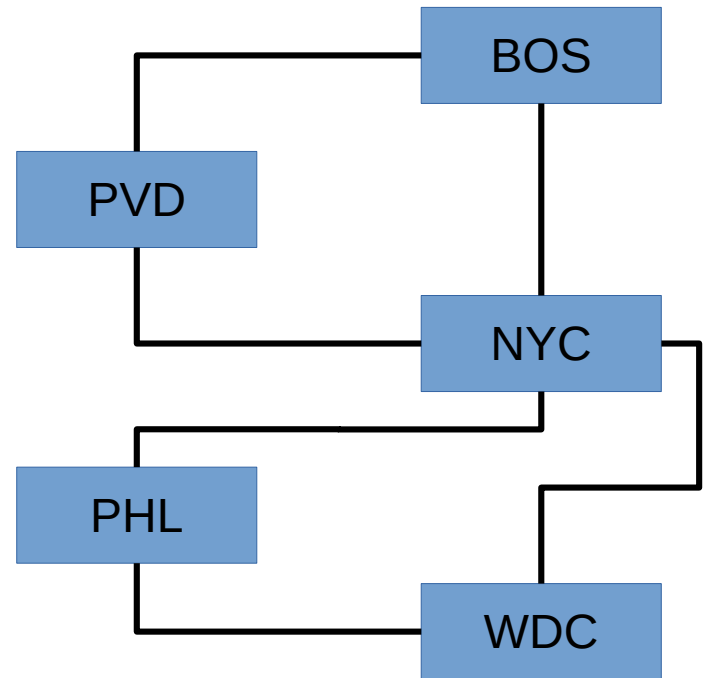
NetworkX

- An Euler circuit is a path through the graph that uses each edge exactly one



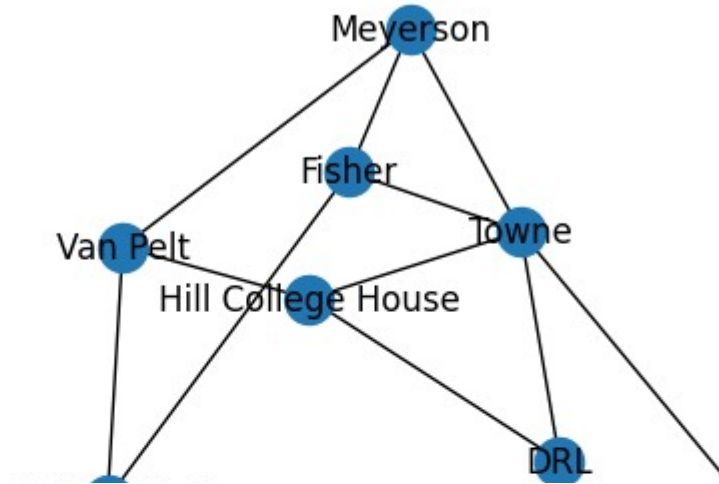
```
list(nx.eulerian_circuit(G))
```

```
[('BOS', 'NYC'),  
 ('NYC', 'WDC'),  
 ('WDC', 'PHL'),  
 ('PHL', 'NYC'),  
 ('NYC', 'PVD'),  
 ('PVD', 'BOS')]
```



NetworkX

- Snow



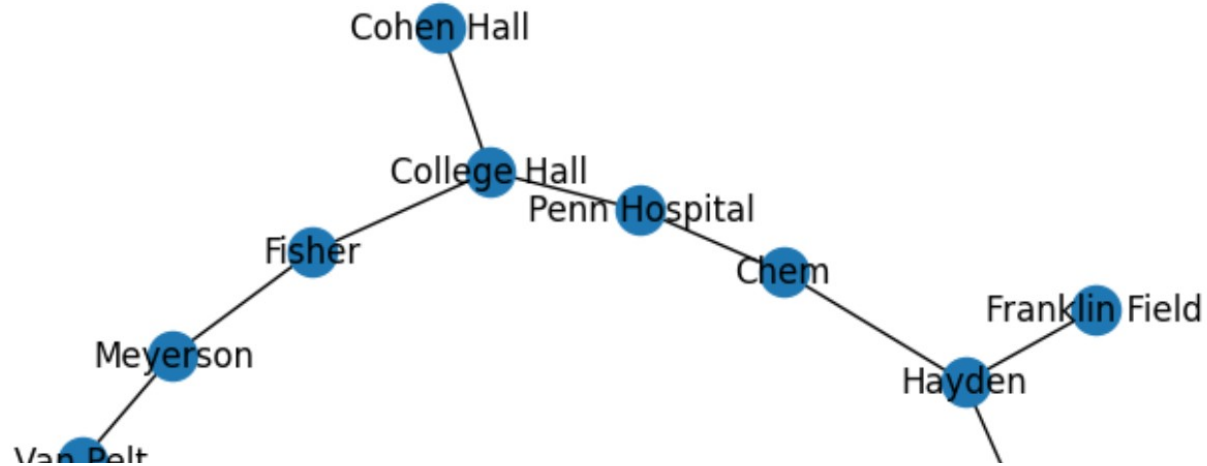
```
weight=3)
)
weight=4)
se", weight=2)
', weight=1)
Pelt", weight=3)
)
nt=1)
```

```
[53] sum(w["weight"] for u, v, w in G.edges(data=True))
```

```
▶ mst = nx.minimum_spanning_tree(G)
   nx.draw(mst, with_labels=True)
```



- Snow-s



✓
0s

```
[58] sum(w["weight"] for u, v, w in mst.edges(data=True))
```

14

```
[65] nx.minimum_cut(G, "College Hall", "Franklin Field", capacity="weight")  
  
(5,  
 ({'Cohen Hall', 'College Hall'},  
  {'Chem',  
   'DRL',  
   'Fisher',  
   'Franklin Field',  
   'Hayden',  
   'Hill College House',  
   'Meyerson',  
   'Penn Hospital',  
   'Towne',  
   'Van Pelt'}))
```

